SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – DESIGN TASK

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 3 – Design Task
Weightage:	40% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	V Yogeswar	**Reg. No.:**	192425175
Section / Batch:		**Date:**	

Code of Conduct

I V Yogeswar / 192425175 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: V Yogeswar

Instructions

- Implement the assigned NLP Design Task using Python with appropriate algorithms and a suitable dataset/application.
- Execute the program and demonstrate the output using relevant sample inputs and test cases.
- Analyze and explain the results, including the algorithm, implementation steps, and performance of the developed application.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
N-gram Based Next-Word Prediction for Smart Text Input	Corpus preprocessing and tokenization Unigram, bigram and trigram counting Probability calculation Next-word prediction	Q1
Smoothing and Backoff Model for Speech/Text Prediction	N-gram frequency calculation Unsmoothed probability estimation Backoff mechanism	Q2
Entropy-Based Evaluation of an English Language Model	Training and testing corpus creation Unigram/bigram/trigram construction Probability estimation	Q3
Part-of-Speech Tagging System for Text Analysis	Text preprocessing and tokenization English/Penn Treebank tagset representation Rule-based POS tagging	Q4

Annexure A – Design Task

Task 1 – N-gram Based Text Prediction

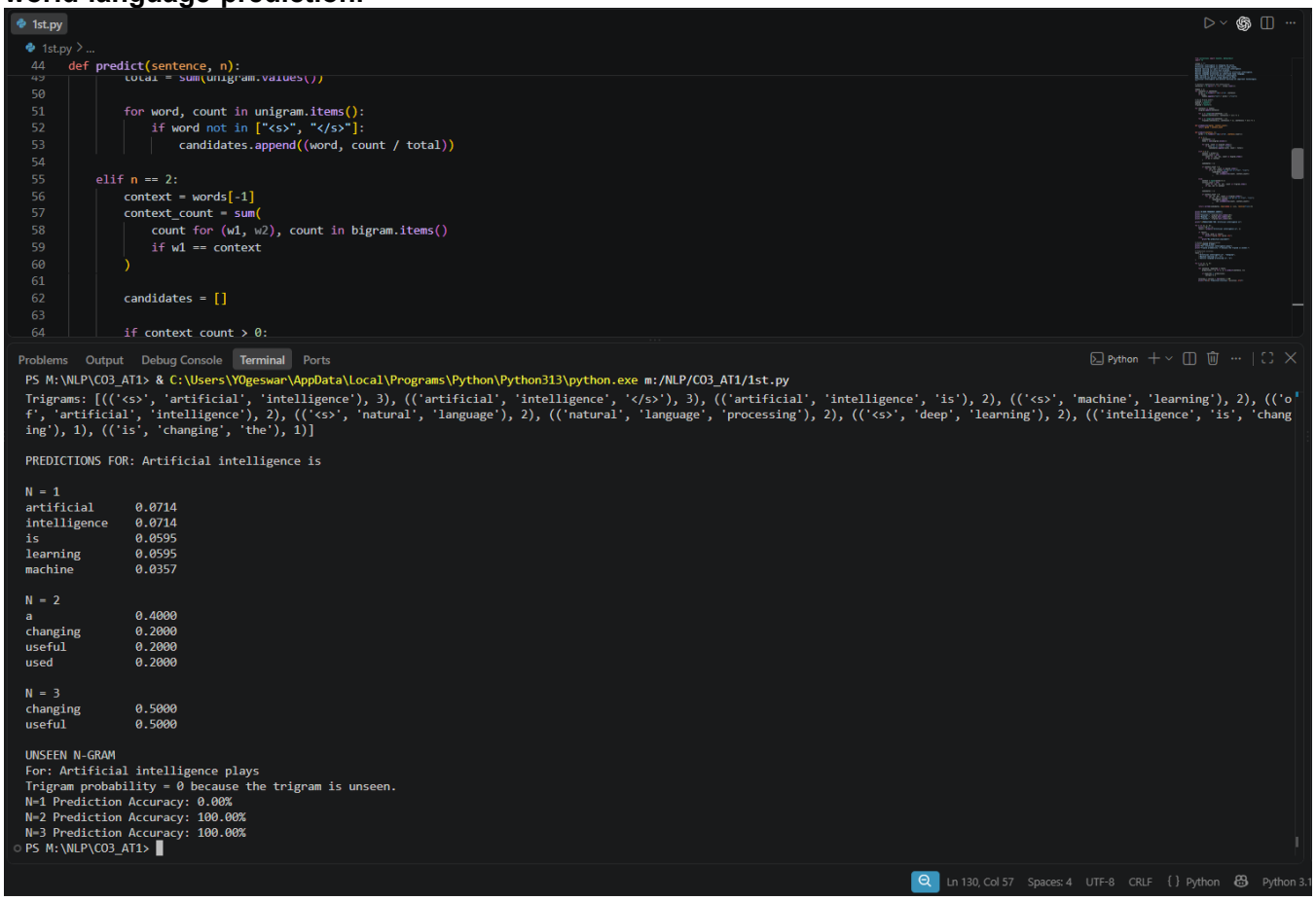
Design and implement a Python-based N-gram language model that predicts the next word in a given sentence using an English text corpus. The system should preprocess the corpus, perform sentence segmentation and tokenization, and construct unigram, bigram, and trigram models by calculating word-frequency counts and maximum-likelihood probabilities.

For an incomplete sentence such as:

"Artificial intelligence is"

the system should identify and display the top-5 most probable next words.

The program should allow the user to select N = 1, 2, or 3, display the corresponding N-gram frequency counts and probabilities, and demonstrate the behavior of the model when an unseen N-gram is encountered. Finally, test the model using multiple incomplete sentences, calculate the prediction accuracy, and discuss the limitations of using an unsmoothed N-gram model for real-world language prediction.



Task 2 – Backoff and Interpolation for Language Prediction

Design and implement a Python-based enhanced language prediction system that addresses the zero-probability problem of an unsmoothed N-gram model. The system should construct unigram, bigram, and trigram models from an English corpus and accept an incomplete sentence such as:

"Machine learning can"

When the required trigram is unavailable, the system should use a backoff strategy by first checking the trigram model, then the bigram model, and finally the unigram model. Extend the system using Deleted Interpolation to combine unigram, bigram, and trigram probabilities using predefined interpolation weights.

The system should generate the top-5 next-word predictions using:

- **Unsmoothed N-gram model**
- **Backoff model**
- **Deleted Interpolation model**

Compare the three approaches in terms of zero probabilities, prediction coverage, and prediction accuracy, and explain why backoff and interpolation are useful for sparse language data.

1st.py 2nd.py

2nd.py > ...

```
def interpolation_predict(sentence):
    words = re.findall(r'\b[a-z]+\b', sentence.lower())
    w1, w2 = words[-2:]

    result = []

    for word in unigram:
        if word in ["<s>", "</s>"]:
            continue

        p1 = unigram_prob(word)
        p2 = bigram_prob(w2, word)
        p3 = trigram_prob(w1, w2, word)

        # Deleted interpolation weights
        p = 0.2 * p1 + 0.3 * p2 + 0.5 * p3
```

Problems Output Debug Console Terminal Ports

```
PS M:\NLP\CO3_AT1> & C:\Users\YOgeswar\AppData\Local\Programs\Python\Python313\python.exe m:/NLP/CO3_AT1/2nd.py
PREDICTIONS FOR: Machine learning can

1. UNSMOOTHED N-GRAM
solve           0.5000
process         0.5000

2. BACKOFF MODEL
solve           0.5000
process         0.5000
learn           0.2000
understand      0.2000
can             0.0676

3. DELETED INTERPOLATION
solve           0.3754
process         0.3127
learn           0.0627
understand      0.0627
can             0.0135

COMPARISON
Unsmoothed : Zero probability for unseen trigrams.
Backoff    : Uses trigram, then bigram, then unigram.
Interpolation: Combines unigram, bigram and trigram probabilities.
PS M:\NLP\CO3_AT1>
```

Task 3 – Entropy-Based Evaluation of Language Models

Design and implement a Python program for measuring the uncertainty of N-gram language models using entropy. Given separate training and testing corpora, construct unigram, bigram, and trigram language models and calculate the probabilities assigned to words in the test sentences. For each test sentence, calculate its entropy and classify the sentence as having relatively high or low predictive uncertainty.

For example, compare sentences such as:
"The cat sits on the mat."
and
"The quantum processor redesigned the..."

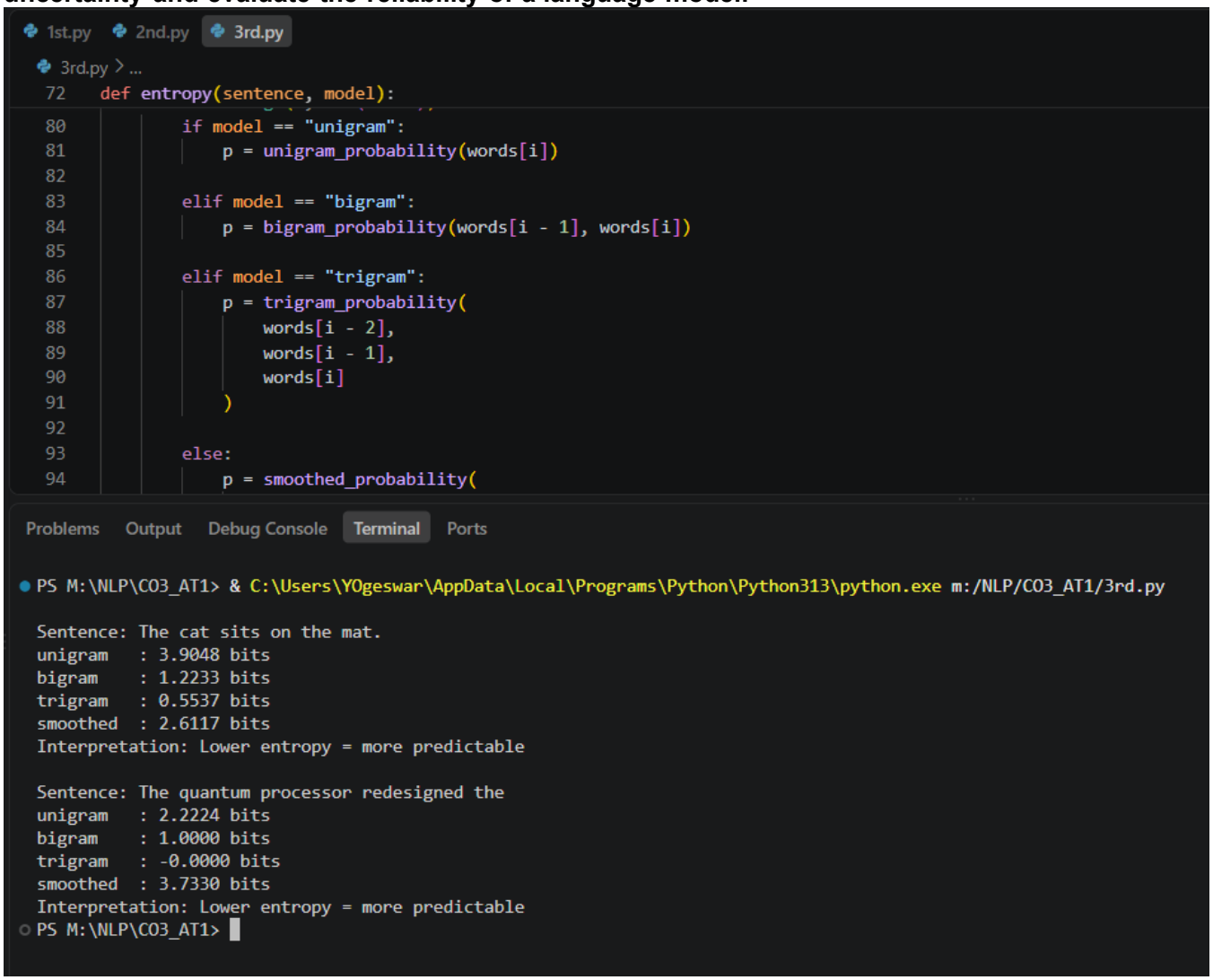
The system should evaluate how predictable the next word is based on the trained language model.

The program should also compare entropy values obtained using:

- **Unigram model**
- **Bigram model**
- **Trigram model**

- **Smoothed model**

Finally, interpret the results and explain how entropy can be used to measure prediction uncertainty and evaluate the reliability of a language model.



Task 4 – Comparative POS Tagging System

Design and implement a Python-based POS tagging system that assigns grammatical tags to words in English sentences.

The system should implement three approaches:

1. Rule-Based POS Tagging

Use a combination of lexical dictionaries, word patterns, and grammatical rules to assign POS tags.

2. Stochastic POS Tagging

Construct a statistical POS tagger using:

- **Word/tag frequencies**
- **Tag transition probabilities**
- **Probabilities obtained from a training corpus**

3. Transformation-Based Tagging

Initially assign tags using a simple tagging strategy and subsequently apply transformation rules to correct likely tagging errors.

For example:

"I book a ticket."

and

"I read a book."

The system should demonstrate how the same word can receive different POS tags depending on its context. Use an appropriate English corpus such as the Brown Corpus or another publicly available tagged corpus. The program should accept user-entered sentences, display the POS tags produced by all three approaches, and compare their results using suitable evaluation

measures such as tagging accuracy. Finally, analyze the differences among the three approaches and explain the advantages and limitations of rule-based, stochastic, and transformation-based POS tagging.

1st.py 2nd.py 3rd.py 4th.py

4th.py > ...

```
160  def accuracy(predicted, actual):
172              total += 1
173
174      return correct / total * 100
175
176
177  # Test sentences
178  sentences = [
179      "I book a ticket",
180      "I read a book",
181      "The cat sits on the mat"
182  ]
183
184  for sentence in sentences:
185
186      words = sentence.split()
```

Problems Output Debug Console Terminal Ports

```
PS M:\NLP\CO3_AT1> & C:\Users\YOgeswar\AppData\Local\Programs\Python\Python313\python.exe m:/NLP/CO3_AT1/4th.py
[('I', 'PRON'), ('book', 'VERB'), ('a', 'DET'), ('ticket', 'NOUN')]

Stochastic:
[('I', 'PRON'), ('book', 'NOUN'), ('a', 'DET'), ('ticket', 'NOUN')]

Transformation-Based:
[('I', 'PRON'), ('book', 'VERB'), ('a', 'DET'), ('ticket', 'NOUN')]

Sentence: I read a book

Rule-Based:
[('I', 'PRON'), ('read', 'NOUN'), ('a', 'DET'), ('book', 'NOUN')]

Stochastic:
[('I', 'PRON'), ('read', 'VERB'), ('a', 'DET'), ('book', 'NOUN')]

Transformation-Based:
[('I', 'PRON'), ('read', 'NOUN'), ('a', 'DET'), ('book', 'NOUN')]

Sentence: The cat sits on the mat

Rule-Based:
[('The', 'DET'), ('cat', 'NOUN'), ('sits', 'NOUN'), ('on', 'ADP'), ('the', 'DET'), ('mat', 'NOUN')]

Stochastic:
[('The', 'DET'), ('cat', 'NOUN'), ('sits', 'VERB'), ('on', 'ADP'), ('the', 'DET'), ('mat', 'NOUN')]

Transformation-Based:
[('The', 'DET'), ('cat', 'NOUN'), ('sits', 'NOUN'), ('on', 'ADP'), ('the', 'DET'), ('mat', 'NOUN')]
```

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25%	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30%	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools	20%	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15%	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10%	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1	3	3	3	3	3	75	75
2	3	3	3	3	3	75	
3	3	3	3	3	3	75	
4	3	3	3	3	3	75	

Faculty In-charge	Course Coordinator	Program Director
KUMARAGURUBARAN. T		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____